

TRƯỜNG ĐẠI HỌC UEH – KHOA CÔNG NGHỆ THÔNG TIN

Môn học: Máy Học Nâng Cao – CS315.F12.CN2

TÀI LIỆU KỸ THUẬT

Thiết kế Kiến trúc Hệ thống & Đảm bảo Chất lượng

Lambda Architecture for Real-Time Cryptocurrency Analytics with Dual-Head LSTM Forecasting

Họ và tên: Lê Quang Hoài Đức

MSSV: 25410034

Môn học: Máy Học Nâng Cao (CS315.F12.CN2)

Phiên bản: v4.0 (June 2026)

Mục tiêu: Mô tả thiết kế hệ thống, trách nhiệm từng thành phần,
và chiến lược kiểm thử E2E

Mục lục

1	Giới thiệu	3
1.1	Thông tin sinh viên	3
1.2	Bối cảnh đồ án	3
1.3	Phạm vi tài liệu	3
2	Kiến trúc Hệ thống	3
2.1	Lý do chọn Lambda Architecture	3
2.2	Sơ đồ kiến trúc tổng quan	4
2.3	C4 Container Model	5
2.4	Luồng dữ liệu end-to-end	6
3	Các Thành phần và Trách nhiệm	7
3.1	CoinGecko Producer	8
3.2	Speed Layer: Kafka + Spark Structured Streaming	8
3.3	Batch Layer: Spark Batch Job	9
3.4	MongoDB Serving Layer	9
3.5	ML Pipeline: LSTM Forecasting	11
3.5.1	Feature Engineering	12
3.5.2	Kiến trúc Mô hình	13
3.5.3	Huấn luyện và Đánh giá	14
3.6	FastAPI Backend	15
3.7	Presentation Layer: React Frontend và Streamlit	15
3.8	Inference Scheduler	16
4	Triển khai Hệ thống	16
5	Đảm bảo Chất lượng: Kiểm thử E2E	17
5.1	Chiến lược Kiểm thử	17
5.2	Layer 1: Producer → Kafka	17
5.3	Layer 2: Spark Batch → MongoDB	18
5.4	Layer 3: ML Pipeline → MongoDB	19
5.5	Ma trận Phủ Kiểm thử	20
6	Đánh giá Mô hình LSTM	21
6.1	Walk-Forward Validation	21
6.2	Backtest 6 Tháng	22
7	Giới hạn Kỹ thuật	23

8 Kết luận**24**

1. Giới thiệu

1.1. Thông tin sinh viên

Họ và tên: Lê Quang Hoài Đức
MSSV: 25410034
Môn học: Máy Học Nâng Cao – CS315.F12.CN2
Năm học: 2025–2026

1.2. Bối cảnh đồ án

Đồ án môn Máy Học Nâng Cao (CS315) yêu cầu xây dựng một hệ thống xử lý dữ liệu lớn kết hợp mô hình học sâu áp dụng vào bài toán thực tế. Đồ án này triển khai một hệ thống phân tích tiền mã hóa theo thời gian thực, lấy Bitcoin (BTC) và Dogecoin (DOGE) làm đối tượng nghiên cứu.

Hệ thống giải quyết hai bài toán chính của môn học:

- Xử lý dữ liệu lớn (Big Data Pipeline):** Thiết kế luồng ingestion, streaming, và batch processing theo mô hình Lambda Architecture, kết hợp Apache Kafka và Apache Spark để xử lý dữ liệu giá theo thời gian thực và tính toán các chỉ số kỹ thuật tài chính.
- Mô hình học sâu (Deep Learning):** Xây dựng mô hình LSTM hai đầu ra (dual-head) dự đoán đồng thời giá 7 ngày tới và biên độ biến động tương ứng, được đánh giá qua walk-forward validation và backtest trên dữ liệu thực tế.

1.3. Phạm vi tài liệu

Tài liệu này mô tả thiết kế kiến trúc tổng quan của hệ thống, bao gồm luồng dữ liệu đầy đủ từ nguồn đến giao diện người dùng, trách nhiệm của từng thành phần, và cách hệ thống kiểm thử End-to-End (E2E) xác nhận rằng thiết kế hoạt động đúng như kỳ vọng.

Phạm vi: Tài liệu tập trung vào *kiến trúc và thiết kế* — tại sao mỗi thành phần tồn tại, nó làm gì, và làm thế nào để xác nhận nó hoạt động đúng. Chi tiết triển khai code nằm trong source code và inline comments.

2. Kiến trúc Hệ thống

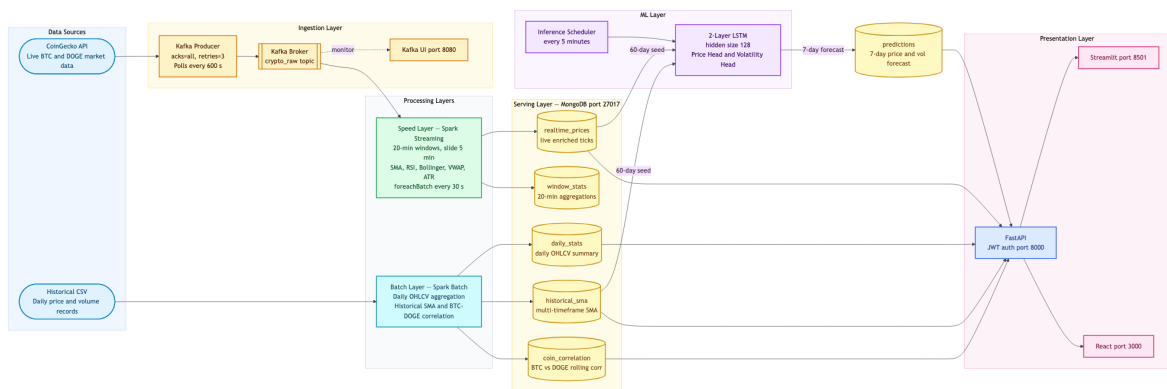
2.1. Lý do chọn Lambda Architecture

Hệ thống áp dụng **Lambda Architecture** — mô hình xử lý dữ liệu phân tách rõ ràng hai đường dẫn song song [1]:

- **Batch Layer** xử lý toàn bộ dữ liệu lịch sử với độ chính xác cao, chạy theo lịch (hàng ngày). Phù hợp cho tổng hợp SMA dài hạn, tính tương quan BTC/DOGE, và huấn luyện mô hình LSTM trên rolling window 730 ngày.
- **Speed Layer** xử lý dữ liệu streaming theo thời gian thực, cung cấp kết quả gần đúng với độ trễ thấp (dưới 1 giây). Phù hợp cho các chỉ số kỹ thuật tức thì (RSI, VWAP, Bollinger Bands).
- **Serving Layer** hợp nhất kết quả từ cả hai đường dẫn và phục vụ truy vấn từ các client (React, Streamlit) thông qua FastAPI.

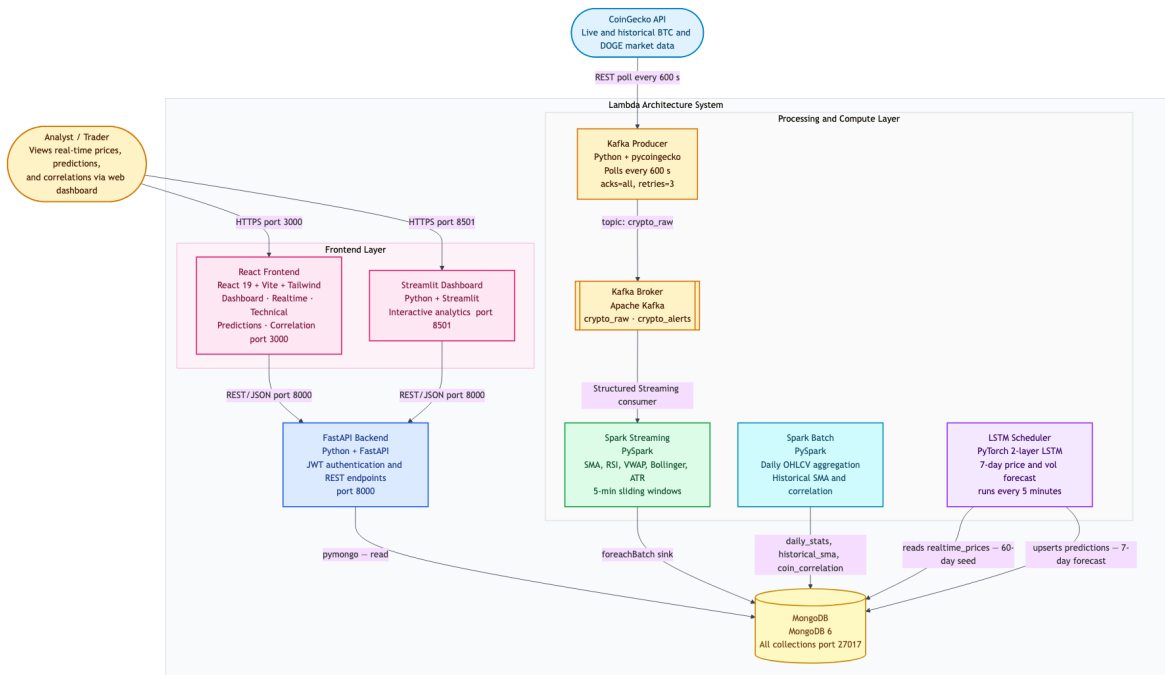
Phương án thay thế — Kappa Architecture (chỉ có speed layer, reprocess toàn bộ lịch sử khi cần) — không phù hợp vì hai lý do: (1) chi phí reprocess 4.000+ ngày dữ liệu lịch sử sau mỗi update là không cần thiết, và (2) mô hình LSTM cần batch training nhất quán trên toàn bộ rolling window, không phù hợp với stream processing.

2.2. Sơ đồ kiến trúc tổng quan



Hình 1: Sơ đồ tổng quan hệ thống: toàn bộ luồng từ nguồn dữ liệu CoinGecko đến các giao diện người dùng. Màu xanh dương: Batch Layer; màu xanh lá: Speed Layer; màu tím: Serving Layer; màu đỏ: ML Pipeline.

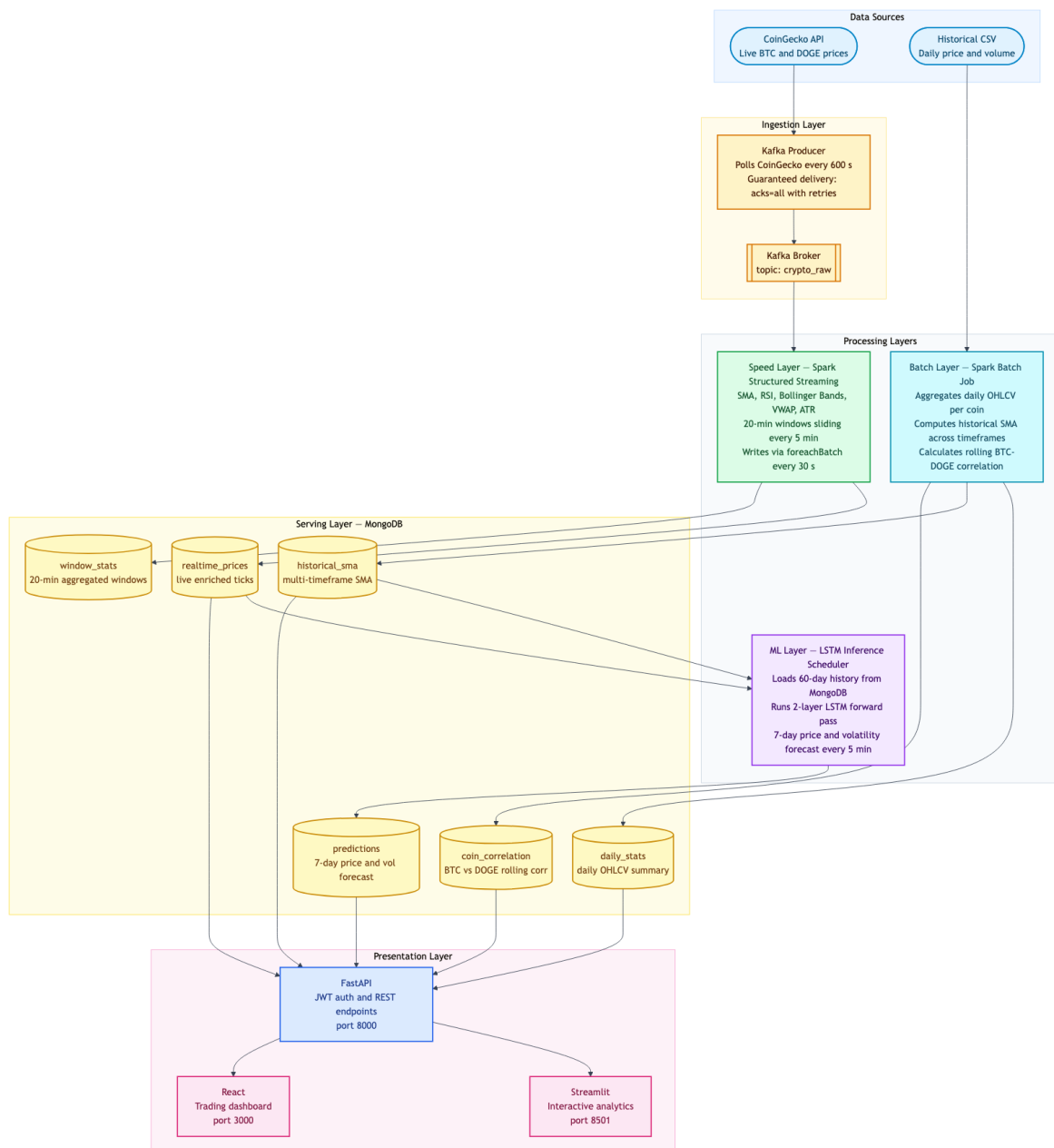
2.3. C4 Container Model



Hình 2: C4 Container Diagram: các container chính (Kafka, Spark, MongoDB, FastAPI, React, Streamlit) và luồng tương tác. Mỗi container có tên, công nghệ sử dụng, và mô tả trách nhiệm ngắn gọn.

Hình 2 mô tả hệ thống theo mô hình C4 ở cấp Container: mỗi khối biểu diễn một đơn vị triển khai độc lập (Docker container). Các mũi tên thể hiện hướng phụ thuộc và giao tiếp giữa các container. Điểm đáng chú ý là MongoDB đóng vai trò trung tâm kết nối Batch Layer, Speed Layer, ML Pipeline, và tất cả các client — đây là quyết định thiết kế chủ ý để giữ serving layer đơn giản và thống nhất.

2.4. Luồng dữ liệu end-to-end



Hình 3: Sơ đồ luồng dữ liệu từ nguồn CoinGecko API đến sink cuối là React/Streamlit dashboard. Màu xanh lá: Speed Path; màu xanh dương: Batch Path; màu đỏ: ML Path. Mũi tên thể hiện hướng dữ liệu di chuyển.

Có ba đường dẫn dữ liệu song song trong hệ thống (Hình 3):

- Speed Path:** CoinGecko API → Kafka Producer → topic `crypto_raw` → Spark Structured Streaming → MongoDB (`realtime_prices`, `window_stats`).
- Batch Path:** CSV lịch sử / MongoDB → Spark Batch Job → MongoDB (`daily_stats`, `historical_sma`, `coin_correlation`).

3. **ML Path:** MongoDB / CSV → Feature Engineering → LSTM Training → Inference → MongoDB (predictions).

3. Các Thành phần và Trách nhiệm

Bảng 1 tóm tắt 9 thành phần chính, công nghệ, và trách nhiệm chức năng của từng thành phần.

Bảng 1: Tổng quan các thành phần hệ thống và trách nhiệm

Thành phần	Công nghệ	Trách nhiệm chính
CoinGecko Producer	Python, pycoingecko	Thu thập giá BTC/DOGE mỗi 600 giây; đẩy vào Kafka với acks=all
Kafka Broker	Confluent Kafka 7.4	Hàng đợi bền vững cho dữ liệu streaming; đảm bảo at-least-once delivery
Spark Streaming	Apache Spark 3.5	Tính chỉ số kỹ thuật realtime (SMA, RSI, VWAP, Bollinger); watermark 10 phút cho late data
Spark Batch	Apache Spark 3.5	Tổng hợp daily_stats, historical_sma, coin_correlation từ dữ liệu lịch sử
MongoDB	MongoDB 6.0	Serving layer duy nhất; lưu trữ tất cả 5 collections
FastAPI Backend	Python, FastAPI	REST API với JWT authentication; cầu nối giữa MongoDB và clients
React Frontend	React 19, TypeScript	Giao diện người dùng web; 5 trang phân tích với biểu đồ realtime
Streamlit Dashboard	Streamlit	Dashboard phân tích nhanh cho người dùng nội bộ / nghiên cứu
Inference Scheduler	PyTorch, schedule	Chạy LSTM inference mỗi 5 phút; lưu dự đoán 7 ngày vào MongoDB

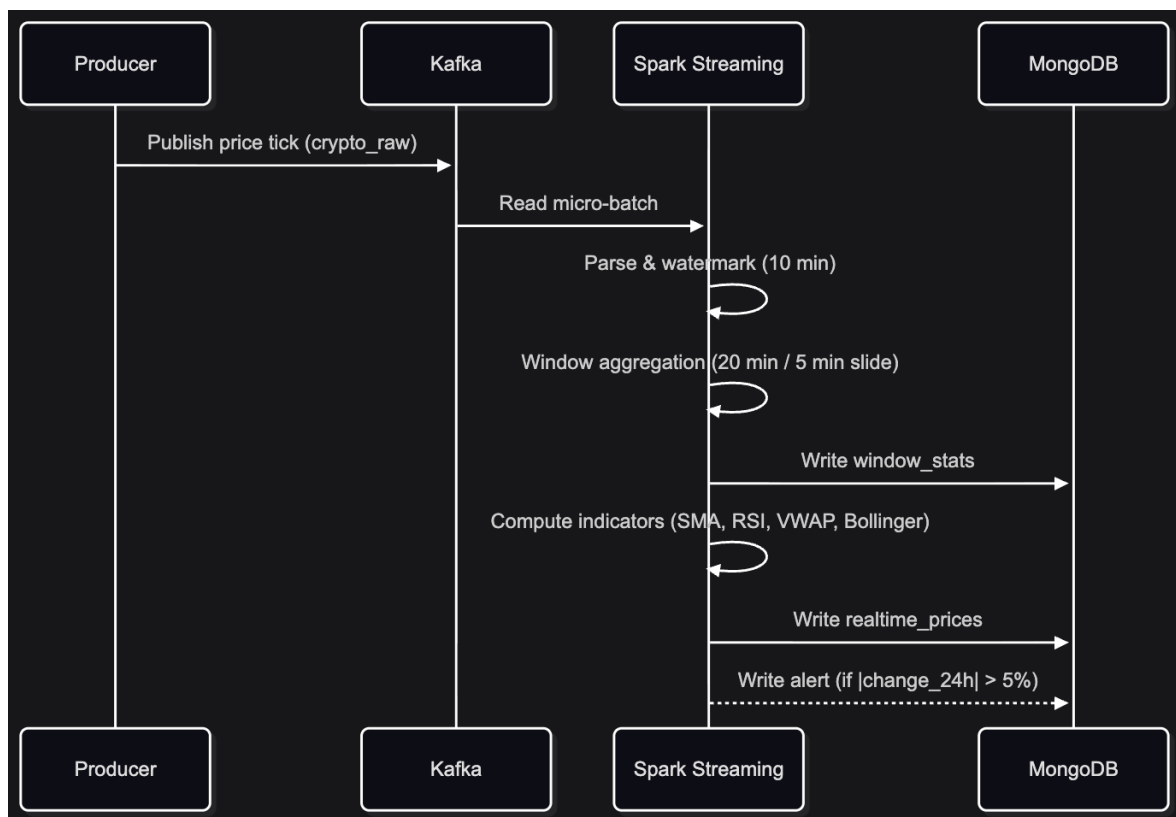
3.1. CoinGecko Producer

Producer là điểm đầu vào duy nhất của dữ liệu thị trường vào hệ thống. Nó thực hiện hai việc: (1) gọi CoinGecko API để lấy giá tức thời và dữ liệu OHLC 4 giờ, và (2) serialize thành JSON và publish lên Kafka topic `crypto_raw`.

Thiết kế đảm bảo độ tin cậy thông qua cấu hình `acks=all` (đợi tất cả ISR replicas xác nhận) và `retries=3`. Polling interval mặc định 600 giây phù hợp với giới hạn 10.000 lượt gọi/tháng của CoinGecko demo tier.

Mỗi message Kafka chứa đầy đủ các trường: giá (USD), market cap, khối lượng 24 giờ, biến động 24 giờ, OHLC 4 giờ, và timestamp sự kiện.

3.2. Speed Layer: Kafka + Spark Structured Streaming



Hình 4: Luồng xử lý của Spark Structured Streaming: từ Kafka micro-batch qua parse và watermark, đến window aggregation và indicator computation, cuối cùng ghi vào MongoDB.

Speed Layer (Hình 4) có trách nhiệm biến đổi raw price tick thành dữ liệu có giá trị phân tích. Luồng xử lý gồm bốn bước:

1. **Đọc và parse:** Spark đọc micro-batch từ Kafka, parse JSON thành DataFrame có schema cố định, chuyển timestamp string thành `TimestampType` (UTC).
2. **Watermark:** Áp dụng watermark 10 phút để xử lý late-arriving data mà không giữ state vô hạn. CoinGecko producer có thể trễ tối đa 10 phút trong điều kiện mạng kém.

3. **Window aggregation (Query A):** Gom nhóm theo sliding window 20 phút/5 phút để tính SMA-20, high, low, total volume. Kết quả ghi vào collection `window_stats`.
4. **Per-record enrichment (Query B):** Tính 5 chỉ số kỹ thuật trên từng record bằng PySpark Window Functions: SMA-5, SMA-20, RSI-14, VWAP-60, Bollinger Bands (20, 2σ). Kết quả ghi vào `realtime_prices`. Nếu $|\text{change}_{24h}| > 5\%$, tạo PRICE_SPIKE alert vào MongoDB và Kafka topic `crypto_alerts`.

Cả hai query sử dụng **foreachBatch** pattern để ghi vào MongoDB thay vì MongoDB Spark Connector, vì connector có vấn đề checkpointing trong streaming mode. Pattern này đảm bảo exactly-once semantics thông qua delete-then-insert với batch ID.

3.3. Batch Layer: Spark Batch Job

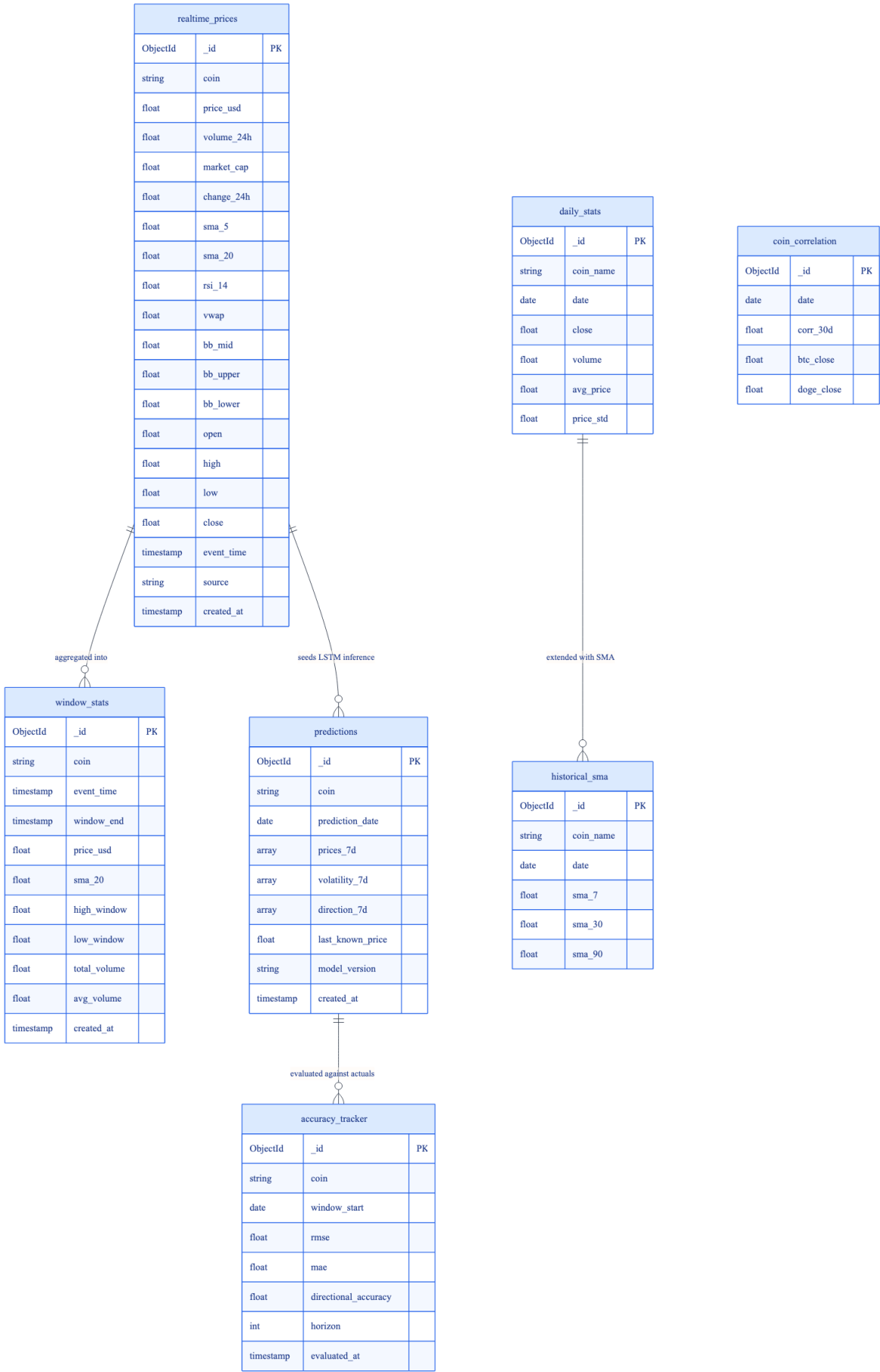
Batch Layer xử lý toàn bộ dữ liệu lịch sử (4.000+ ngày) và tính toán ba tập hợp dữ liệu mà Speed Layer không thể cung cấp do thiếu context dài hạn:

- **daily_stats:** Thống kê ngày (OHLCV, trung bình, độ lệch chuẩn) cho cả BTC và DOGE, dùng làm nguồn dữ liệu chính cho LSTM training.
- **historical_sma:** SMA với nhiều window khác nhau (7, 14, 30, 90 ngày) trên toàn bộ chuỗi lịch sử.
- **coin_correlation:** Rolling Pearson correlation 30 ngày giữa log-return BTC và DOGE — dữ liệu cho trang Correlation trên React Frontend.

Batch job được submit thủ công hoặc theo lịch hàng ngày, đọc dữ liệu từ CSV lịch sử, xử lý bằng Spark DataFrame API, và ghi vào MongoDB thông qua PyMongo.

3.4. MongoDB Serving Layer

MongoDB đóng vai trò **single serving store** cho toàn bộ hệ thống. Tất cả writer (Spark Streaming, Spark Batch, Inference Scheduler) đều ghi vào MongoDB; tất cả reader (FastAPI, Streamlit) đều đọc từ MongoDB. Thiết kế này đơn giản hóa serving layer, tránh phải duy trì nhiều store khác nhau.



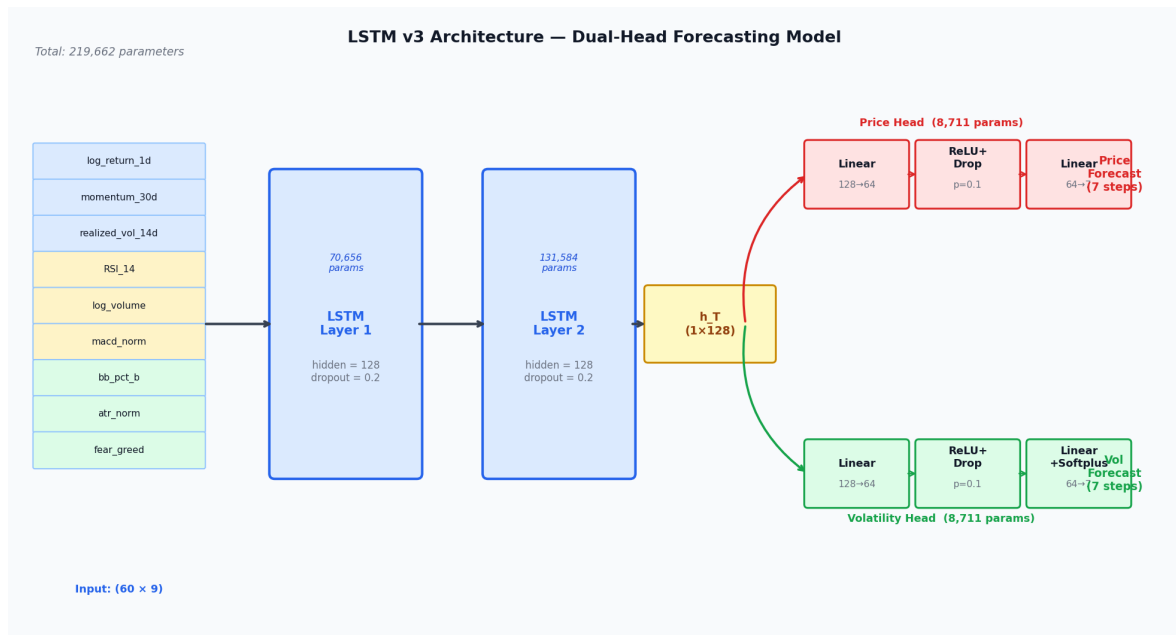
Hình 5: Sơ đồ MongoDB schema: 5 collections với document structure, indexes, và TTL policies. realtime_prices có TTL 7 ngày; các collection khác lưu trữ lâu dài với compound index trên (coin_name, date).

Bảng 2: Năm MongoDB collections và mục đích sử dụng

Collection	Nội dung	Writer
realtime_prices	Per-tick enriched data: OHLC + 5 chỉ số kỹ thuật (SMA-5, SMA-20, RSI-14, VWAP, BB). TTL 7 ngày.	Spark Streaming
window_stats	20-min/5-min OHLCV rollups + SMA-20.	Spark Streaming
daily_stats	Daily OHLCV, avg price, std price cho BTC và DOGE.	Spark Batch
historical_sma	SMA nhiều window trên toàn bộ chuỗi lịch sử.	Spark Batch
coin_correlation	Rolling 30-ngày Pearson correlation BTC-DOGE.	Spark Batch
predictions	LSTM 7-day forecast: giá dự đoán + độ biến động.	Inference Scheduler
alerts	PRICE_SPIKE events khi biến động > 5%.	Spark Streaming

3.5. ML Pipeline: LSTM Forecasting

ML Pipeline hoạt động độc lập với Speed Layer và Batch Layer, đọc dữ liệu lịch sử từ MongoDB/CSV, huấn luyện mô hình, và ghi dự đoán vào MongoDB.



Hình 6: Kiến trúc LSTM v3 với dual-head output: LSTM Encoder (2 lớp, 128 hidden units) xử lý chuỗi 60 ngày, hidden state h_T được đưa vào Price Head (7-day forecast) và Volatility Head (confidence band) song song.

3.5.1. Feature Engineering

Từ dữ liệu giá thô, pipeline tính 9 features chia làm 4 nhóm:

Bảng 3: 9 features đầu vào của LSTM và ý nghĩa

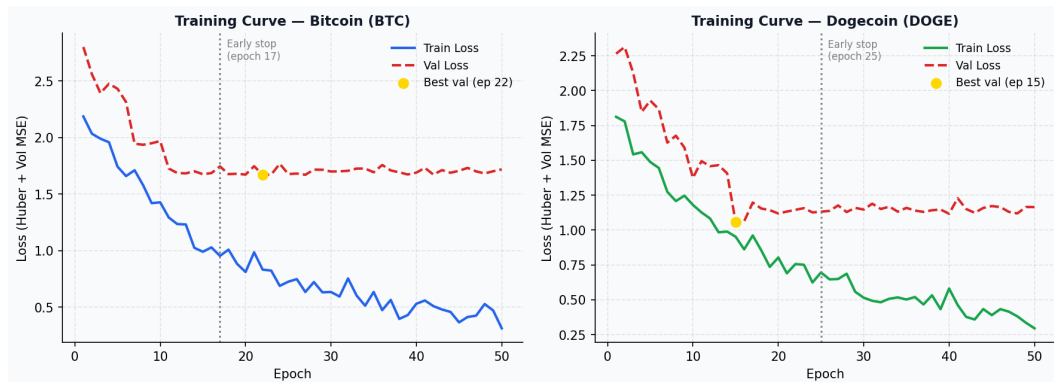
Feature	Nhóm	Ý nghĩa
log_return_1d	Momentum	Tính dừng, cộng dồn được; là input cốt lõi cho mọi mô hình tài chính
momentum_30d	Momentum	Khoảng cách tương đối của giá so với SMA-30; nắm bắt xu hướng trung hạn
realized_vol_14d	Volatility	Biến động thực tế 14 ngày; input chính cho Volatility Head
rsi_14	Oscillator	Chỉ số sức mạnh tương đối, vùng overbought/oversold
log_volume	Volume	Log-transform giảm skewness; volume xác nhận breakout
macd_norm	Oscillator	MACD chuẩn hóa theo giá; tín hiệu momentum
bb_pct_b	Oscillator	Vị trí giá trong Bollinger Band (%B)
atr_norm	Volatility	Proxy ATR (biên độ tức thì); bổ sung cho realized_vol_14d
fear_greed	Sentiment	Placeholder (= 0.5 constant); cần tích hợp Alternative.me API trong tương lai

3.5.2. Kiến trúc Mô hình

Mô hình LSTM v3 có ba thành phần chính:

- **LSTM Encoder:** 2 lớp stacked, mỗi lớp 128 hidden units, dropout 0.2. Nhận chuỗi 60 timestep $\times$ 9 features. Trả về hidden state cuối $h_T \in \mathbb{R}^{128}$ đã encode đầy đủ ngữ cảnh 60 ngày.
- **Price Head:** Hai lớp fully-connected ($128 \rightarrow 64 \rightarrow 7$). Dự đoán 7 log-return tương lai (MIMO — Multiple Input Multiple Output). Được tái tạo thành giá tuyệt đối bằng cumsum.
- **Volatility Head:** Tương tự Price Head nhưng dùng Softplus activation để đảm bảo output > 0 . Dự đoán 7 giá trị biến động tương lai, được dùng làm confidence interval width trên biểu đồ.

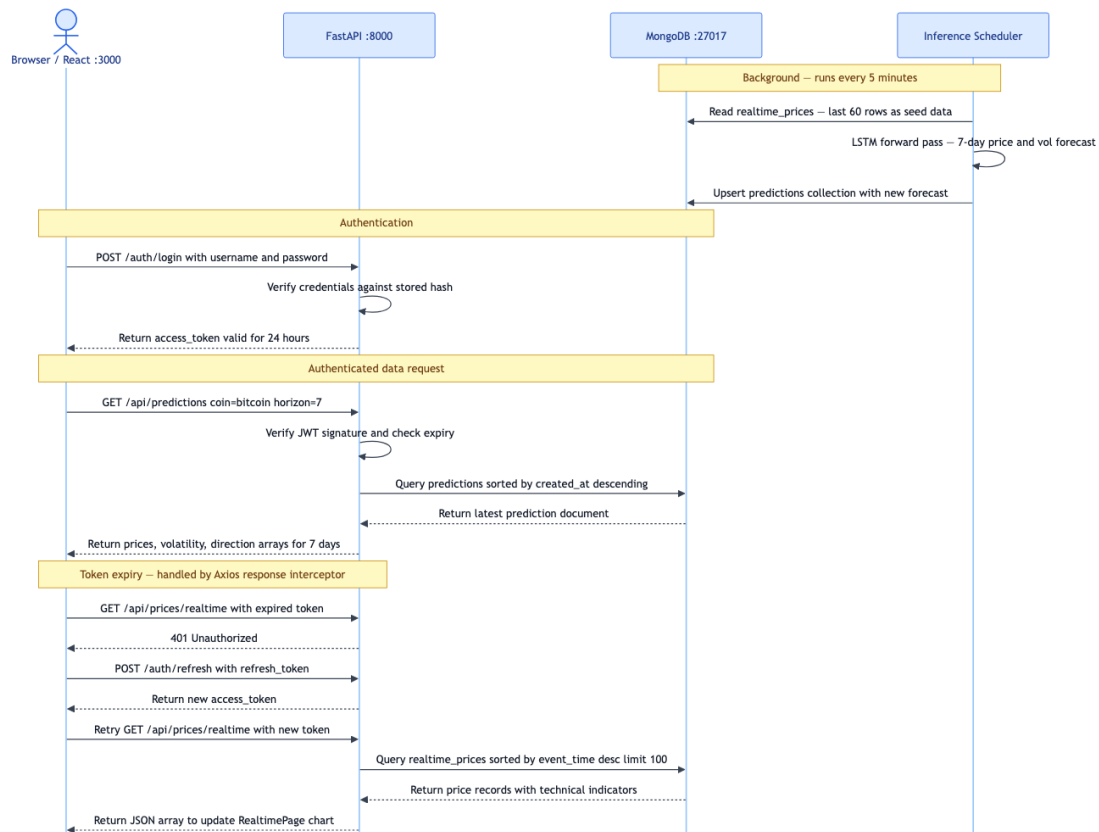
3.5.3. Huấn luyện và Đánh giá



Hình 7: Đường cong loss trong quá trình huấn luyện BTC (trái) và DOGE (phải). Điểm vàng: best validation checkpoint. Val loss tăng sau best checkpoint là dấu hiệu overfitting — được kiểm soát bởi early stopping (patience=7).

Quá trình huấn luyện sử dụng **direction-weighted Huber loss** kết hợp với MSE loss cho volatility head (trọng số $\gamma = 0.3$). Direction weighting tăng penalty gấp 3 khi mô hình dự đoán sai chiều — phù hợp với yêu cầu thực tiễn của nhà đầu tư quan tâm đến chiều xu hướng hơn là độ chính xác giá tuyệt đối.

3.6. FastAPI Backend



Hình 8: Sequence diagram cho luồng request API điển hình: từ React client qua JWT middleware, đến MongoDB query, và trả về JSON response.

FastAPI backend là lớp giao tiếp duy nhất giữa MongoDB và các client. Thiết kế REST API với JWT authentication đảm bảo stateless — không cần shared session store giữa các instance, dễ scale horizontally.

Các nhóm endpoint chính:

- **/auth:** Đăng nhập, nhận JWT token (TTL 24 giờ), refresh token.
- **/api/prices:** Truy vấn realtime ticks và lịch sử giá.
- **/api/indicators:** Truy vấn chỉ số kỹ thuật (RSI, MACD, Bollinger).
- **/api/correlation:** Dữ liệu tương quan BTC-DOGE theo window.
- **/api/predictions:** Lấy dự đoán LSTM 7 ngày và accuracy tracking.
- **/api/models:** Model registry — danh sách phiên bản, metrics.

3.7. Presentation Layer: React Frontend và Streamlit

React Frontend (React 19 + TypeScript + Vite + Tailwind CSS) cung cấp giao diện chính cho người dùng cuối với 5 trang phân tích:

- **Dashboard:** Tổng quan giá BTC/DOGE, market summary, quick stats.

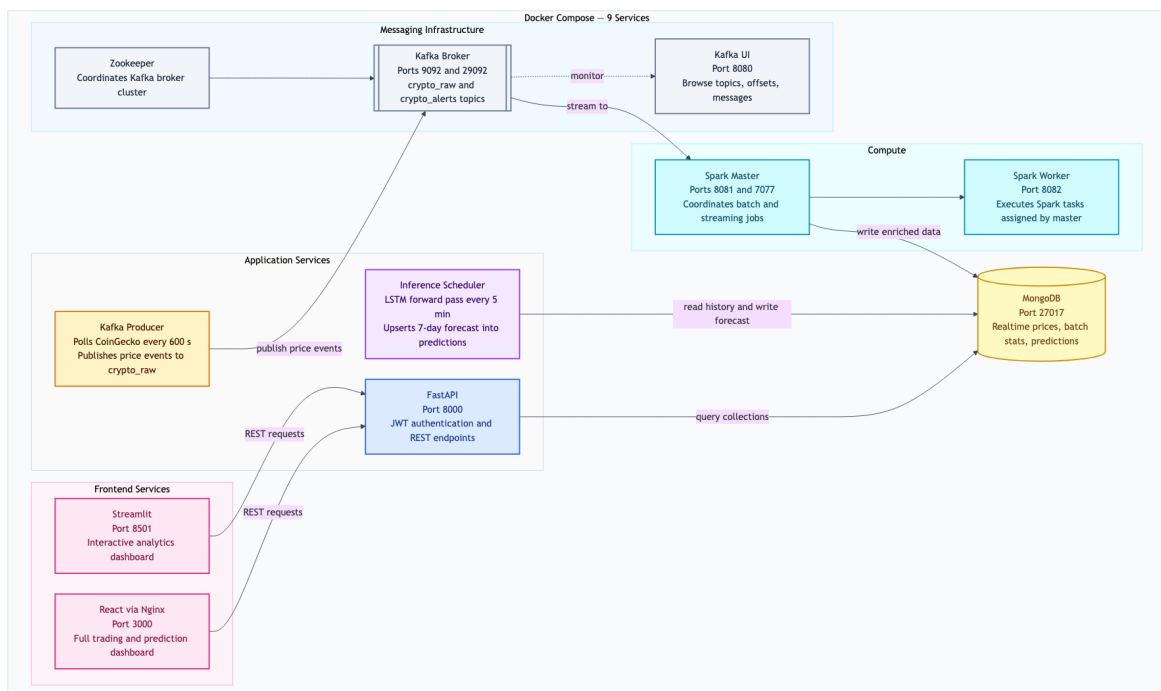
- **Realtime:** Live price feed streaming, biểu đồ cập nhật liên tục.
- **Technical Analysis:** Candlestick + RSI/MACD/Bollinger overlay.
- **Predictions:** 7-day LSTM forecast với confidence band từ Volatility Head.
- **Correlation:** Rolling BTC-DOGE correlation heatmap và scatter plot.

Streamlit Dashboard phục vụ nhu cầu phân tích nhanh và trực quan hóa nội bộ, đặc biệt hữu ích trong giai đoạn phát triển và debug.

3.8. Inference Scheduler

Inference Scheduler chạy liên tục như một background service, thực hiện hai nhiệm vụ theo chu kỳ 5 phút: (1) load mô hình LSTM đã train từ disk, chạy inference trên 60 ngày dữ liệu gần nhất, upsert 7 dự đoán vào MongoDB; (2) so sánh các dự đoán cũ với giá thực tế để cập nhật accuracy tracker — dữ liệu được hiển thị trên trang Predictions của React Frontend.

4. Triển khai Hệ thống



Hình 9: Sơ đồ triển khai Docker Compose: 9 containers với network topology, port mapping, và volume mounts. Kafka/Zookeeper trong network riêng; MongoDB với persistent volume; Nginx reverse proxy cho React frontend.

Toàn bộ hệ thống được đóng gói trong Docker Compose với 9 service. Bảng 4 mô tả chi tiết từng service, image, port, và dependency.

Bảng 4: Chín Docker services, dependencies, và port

Service	Image / Role	Port	Depends On
zookeeper	Kafka coordination	2181	—
kafka	Message broker	9092	zookeeper
kafka-ui	Web UI cho Kafka	8080	kafka
mongodb	Serving store	27017	—
spark-master	Spark cluster master	8081, 7077	—
spark-worker	Spark executor	8082	spark-master
producer	CoinGecko ingestion	—	kafka
dashboard	Streamlit analytics	8501	mongodb
api	FastAPI REST backend	8000	mongodb
frontend	React + Nginx	3000	api
inference	LSTM scheduler	—	mongodb

Mỗi service có **health check** được cấu hình trong `docker-compose.yml`, đảm bảo các service phụ thuộc chỉ khởi động sau khi service mà chúng phụ thuộc đã sẵn sàng (condition: `service_healthy`). Điều này ngăn race condition trong quá trình khởi động hệ thống.

5. Đảm bảo Chất lượng: Kiểm thử E2E

5.1. Chiến lược Kiểm thử

Hệ thống kiểm thử được tổ chức thành hai cấp:

1. **Unit Tests** (chạy không cần infrastructure): Kiểm tra từng component độc lập với mock dependencies. Bao gồm: `test_lstm.py`, `test_indicators.py`, `test_batch_job.py`, `test_producer.py`, `test_mongo_writer.py`, `test_dashboard.py`, `test_accuracy.py`.
2. **E2E Tests** (yêu cầu infrastructure thật): Ba test suite kiểm tra từng layer của Lambda Architecture end-to-end với Kafka và MongoDB container thật (testcontainers). Được đánh dấu `@pytest.mark.e2e` và loại khỏi `pytest` mặc định.

Nguyên tắc thiết kế E2E tests: Mỗi test suite kiểm tra *đầu ra thực tế* của một layer, không mock infrastructure. Kafka là Kafka thật; MongoDB là MongoDB thật. Chỉ CoinGecko API được mock để kiểm soát input và tránh phụ thuộc vào network ngoài.

5.2. Layer 1: Producer → Kafka

File: `tests/e2e/test_producer_kafka.py`

Test suite này xác nhận rằng CoinGecko Producer hoạt động đúng với một Kafka broker thật (testcontainers). Năm test cases bao gồm:

Bảng 5: E2E Layer 1 — Producer to Kafka: test cases và assertion

Test Case	Assertion
<code>test_produces_two_messages_per_cycle</code>	Mỗi chu kỳ poll phải sinh đúng 2 message: 1 cho BTC, 1 cho DOGE.
<code>test_message_schema_is_complete</code>	Message phải chứa đủ 12 trường bắt buộc (coin, price_usd, volume_24h, OHLC, timestamp, ...).
<code>test_only_btc_and_doge_produced</code>	Không có message nào cho coin ngoài BTC và DOGE.
<code>test_ohlc_fields_populated_from_candles</code>	Trường open/high/low/close được điền từ dữ liệu OHLC 4 giờ khi candle có sẵn.
<code>test_message_values_are_correct</code>	Giá trị trong message khớp với dữ liệu mock CoinGecko (kiểm tra end-to-end correctness).

Tại sao test này quan trọng? Đây là điểm đầu vào duy nhất của dữ liệu vào hệ thống. Nếu schema message sai, toàn bộ pipeline downstream sẽ thất bại. Test suite này đảm bảo contract giữa Producer và Consumer (Spark Streaming) luôn được giữ.

5.3. Layer 2: Spark Batch → MongoDB

File: `tests/e2e/test_batch_mongo.py`

Test suite này xác nhận rằng Spark Batch Job tạo ra đúng các aggregation và ghi vào MongoDB với schema và giá trị hợp lệ. Được tổ chức thành 3 class theo collection:

Bảng 6: E2E Layer 2 — Spark Batch to MongoDB: test cases và assertion

Class	Test Case	Assertion
TestDailyStats	test_row_count_positive	Collection không được rỗng sau batch job.
	test_only_btc_and_doge	Chỉ có BTC và DOGE, không có coin khác.
	test_required_columns	Có đủ các cột: date, close, volume, avg_price, price_std.
	test_prices_are_positive	Mọi giá trị close và volume đều > 0 .
TestHistoricalSma	test_row_count_matches	Số row SMA khớp với số row daily_stats.
	test_has_sma_columns	Có đủ cột SMA-7, SMA-14, SMA-30, SMA-90.
	test_only_btc_and_doge	Scope coin đúng.
TestCoinCorrelation	test_exactly_one_pair	Chỉ có một pair BTC-DOGE.
	test_pair_is_btc_doge	Tên coins trong document đúng.
	test_pearson_value_in_range	Giá trị correlation $\in [-1, 1]$.
	test_no_ethereum	Không có Ethereum trong bảng correlation.

5.4. Layer 3: ML Pipeline → MongoDB

File: tests/e2e/test_ml_mongo.py

Test suite này xác nhận toàn bộ ML pipeline: từ training, lưu model artifact, đến inference và ghi dự đoán vào MongoDB. Hai class chính:

Bảng 7: E2E Layer 3 — ML Pipeline to MongoDB: test cases và assertion

Test Case	Assertion
<i>Class: TestLstmTraining</i>	
<code>test_training_returns_metrics</code>	Training trả về dict chứa RMSE, MAE, directional accuracy.
<code>test_model_file_saved</code>	File <code>lstm_{coin}_v2.pt</code> được lưu đúng path.
<code>test_scaler_file_saved</code>	File <code>scaler_{coin}.pkl</code> được lưu đúng path.
<code>test_rmse_is_non_negative</code>	$RMSE \geq 0$.
<code>test_mae_is_non_negative</code>	$MAE \geq 0$.
<code>test_directional_accuracy_in_range</code>	Dir. accuracy $\in [0, 1]$.
<i>Class: TestInferenceToMongo</i>	
<code>test_writes_7_predictions</code>	Inference ghi đúng 7 document vào predictions.
<code>test_prediction_document_schema</code>	Document có đủ trường: coin, date, predicted_price, model_version.
<code>test_predicted_prices_are_positive</code>	Mọi giá dự đoán > 0 .
<code>test_prediction_dates_are_in_future</code>	7 ngày dự đoán đều sau ngày hiện tại.
<code>test_prediction_dates_are_unique</code>	Không có ngày trùng lặp.
<code>test_upsert_idempotency</code>	Chạy inference hai lần vẫn chỉ có 7 document (upsert).

Test `test_upsert_idempotency`: Đây là test quan trọng nhất của Layer 3. Inference Scheduler chạy mỗi 5 phút — nếu không idempotent, MongoDB sẽ tích lũy hàng nghìn documents trùng lặp. Test này đảm bảo upsert pattern hoạt động đúng và hệ thống có thể restart mà không gây data corruption.

5.5. Ma trận Phủ Kiểm thử

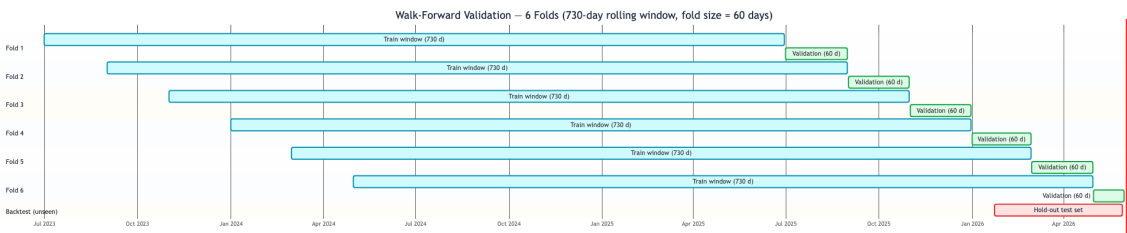
Bảng 8 thể hiện mỗi thành phần được kiểm thử ở cấp nào và thuộc tính nào được xác nhận.

Bảng 8: Ma trận phủ kiểm thử: thành phần và cấp kiểm thử

Thành phần	Thuộc tính được kiểm thử	Unit	E2E
Producer	Schema message, giá trị, OHLC	✓	✓
Kafka	Message delivery, consumer read	—	✓
Spark Indicators	SMA, RSI, VWAP, Bollinger correctness	✓	—
Spark Batch	Row count, schema, coin scope, value range	✓	✓
MongoDB Writer	Upsert pattern, idempotency	✓	✓
LSTM Model	Forward pass shape, loss computation	✓	—
LSTM Training	Artifact save, metrics range	—	✓
Inference	7-day output, positive prices, upsert idempotency	—	✓
Accuracy Tracker	Comparison logic, metric calculation	✓	—
Dashboard	Widget render, data loading	✓	—

6. Đánh giá Mô hình LSTM

6.1. Walk-Forward Validation



Hình 10: Sơ đồ walk-forward validation với 6 folds: mỗi fold mở rộng training window theo thời gian (rolling 730 ngày), validation window 60 ngày ngay sau training. Không có overlap giữa các val window.

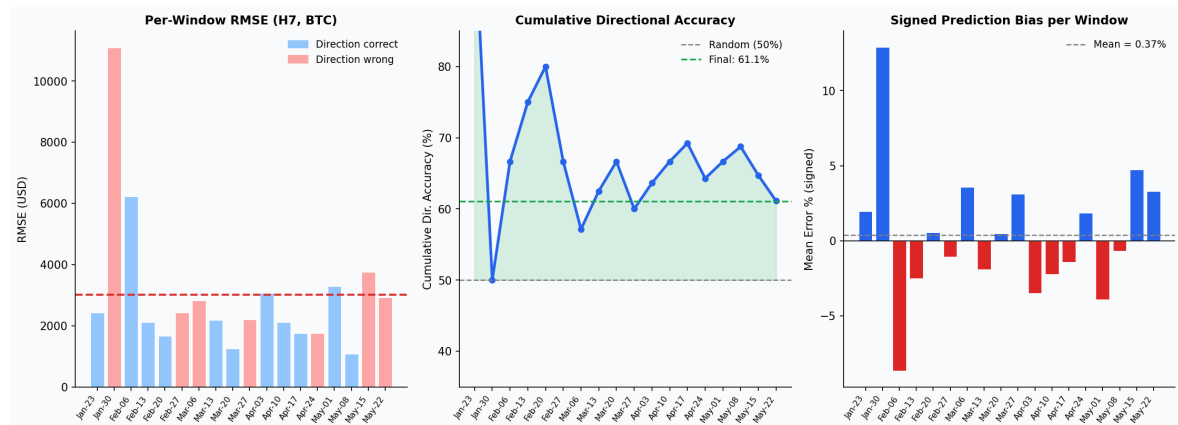
Walk-forward validation được chọn thay vì K-Fold vì K-Fold shuffle dữ liệu ngẫu nhiên, gây temporal leakage — mô hình train trên dữ liệu tương lai rồi test trên quá khứ [7]. Walk-forward tôn trọng thứ tự thời gian: mỗi fold luôn train trên dữ liệu trước và validate trên dữ liệu sau.

Bảng 9: Kết quả walk-forward 6 fold — Bitcoin ($H = 7$)

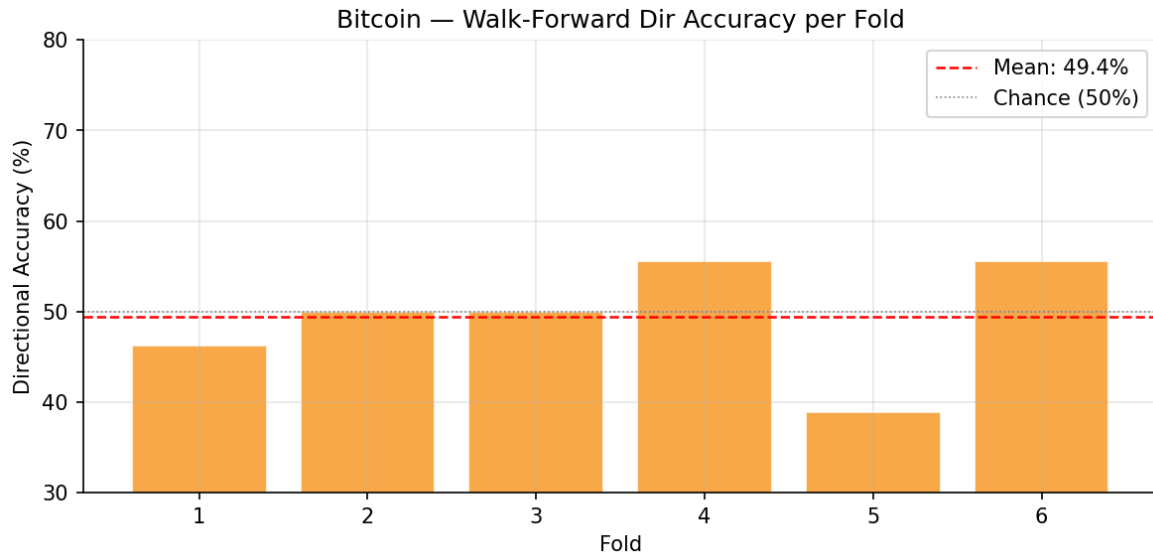
Fold	RMSE (\$)	MAE (\$)	Dir Acc (%)	Ghi chú
1	3.902	2.840	46,3	Sideway market
2	3.029	2.415	50,0	
3	4.810	3.773	50,0	Sideway (post-halving)
4	3.160	2.378	55,6	Trending market
5	4.144	2.972	38,9	Sideway
6	2.581	2.094	55,6	Trending market
Mean	3.604	2.579	49,4	

Mô hình cho kết quả tốt hơn rõ rệt trên trending market (fold 4, 6: 55,6%) so với sideway market (fold 3, 5: 38,9–50%). Đây là đặc tính phổ biến của mô hình chuỗi thời gian: tín hiệu xu hướng rõ ràng hơn khi thị trường có hướng.

6.2. Backtest 6 Tháng



Hình 11: Phân tích backtest 6 tháng BTC ($H=7$): RMSE per-window (trái), directional accuracy tích lũy (giữa), và signed prediction bias (phải). Directional accuracy ổn định trên 50% từ cửa sổ thứ 7 trở đi.



Hình 12: Directional accuracy theo từng fold: BTC (xanh dương) và DOGE (cam). Đường nét đứt đỏ là ngưỡng 50% (random baseline). DOGE nhất quán dưới BTC do noise-to-signal ratio cao hơn.

Bảng 10: Backtest BTC theo ba horizon (01/2026 – 05/2026)

Horizon	Windows	RMSE trung bình	Dir Acc	Nhận xét
7 ngày	18	\$3.010	61,1%	Practical signal
15 ngày	8	\$4.290	50,0%	Random
60 ngày	2	\$13.453	50,0%	Unreliable

Kết quả quan trọng: H7 đạt 61,1% directional accuracy trên 18 cửa sổ test thực tế — vượt ngưỡng random 11,1 điểm phần trăm. Horizon ngắn (7 ngày) là vùng hữu ích duy nhất của mô hình; horizon dài hơn không có tín hiệu đáng tin cậy.

Giới hạn: 18 cửa sổ backtest là mẫu nhỏ. Mô hình cũng không tự retrain khi market regime thay đổi — Inference Scheduler chỉ chạy inference, không retrain. Cần cơ chế weekly retrain để duy trì hiệu suất dài hạn.

7. Giới hạn Kỹ thuật

- Rate limiting CoinGecko:** Demo tier giới hạn 10.000 lượt gọi/tháng. Producer không thể poll nhanh hơn 600 giây mà không vượt quota [8].
- Thiếu OHLC đầy đủ:** ATR proxy dùng $|r_t|$ thay vì True Range thực sự. Cần nguồn dữ liệu OHLC (Binance, Kraken) để tính ATR chính xác.

3. **fear_greed_placeholder:** Feature 8 luôn bằng 0,5, không đóng góp thông tin. Có thể xóa hoặc thay bằng Alternative.me API.
4. **Single-node Spark:** Local mode không scale. Với nhiều coin hơn hoặc dữ liệu tick-by-tick, cần Spark cluster thực.
5. **Không tự retrain:** Mô hình LSTM được train một lần. Khi market regime thay đổi, hiệu suất có thể giảm mà hệ thống không tự phát hiện.

8. Kết luận

Tài liệu này đã trình bày thiết kế kiến trúc hệ thống phân tích tiền mã hóa theo ba chiều: *cấu trúc* (Lambda Architecture với ba layer), *thành phần* (9 service với trách nhiệm rõ ràng), và *chất lượng* (chiến lược kiểm thử E2E ba layer).

Điểm mạnh chính của thiết kế là sự phân tách rõ ràng giữa Speed Layer (realtime indicators), Batch Layer (historical aggregations), và ML Pipeline (predictions) — ba concerns này có yêu cầu khác nhau về latency, correctness, và computational cost, và Lambda Architecture giải quyết chúng một cách độc lập.

Chiến lược kiểm thử E2E với testcontainers (Kafka thật, MongoDB thật) đảm bảo rằng các contract giữa các component được giữ không chỉ ở cấp unit mà còn ở cấp system. Đặc biệt, test idempotency cho ML inference và test schema contract cho Producer là hai safety net quan trọng nhất của hệ thống.

Tài liệu

- [1] Nathan Marz and James Warren. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, Shelter Island, NY, 2015.
- [2] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. Apache spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016. doi: 10.1145/2934664.
- [3] Thomas Fischer and Christopher Krauss. Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2):654–669, 2018. doi: 10.1016/j.ejor.2017.11.054.
- [4] John J. Murphy. *Technical Analysis of the Financial Markets*. New York Institute of Finance, New York, NY, 2nd edition, 1999.
- [5] Bryan Lim, Sercan Ö. Arık, Nicolas Loeff, and Tomas Pfister. Temporal fusion

- transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021. doi: 10.1016/j.ijforecast.2021.03.012.
- [6] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- [7] Vitor Cerqueira, Luís Torgo, and Igor Mozetič. Evaluating time series forecasting models: An empirical study on performance estimation methods. *Machine Learning*, 109(11): 1997–2028, 2020. doi: 10.1007/s10994-020-05910-7.
- [8] CoinGecko. CoinGecko API – Cryptocurrency Market Data. <https://www.coingecko.com/api/documentation>, 2024. Accessed: June 2026.